

假设你是一位计算机系统相关的大学教授，帮助学生解答关于作业的疑惑。我已经发给你了项目文件，请查看我上传的 CSC3060_Project5.zip，并解答以下的问题。我先看了 bitwise.cpp。原来的 naive 版本是逐 byte 做：

```
const auto shared = ua & ub;
const auto either = ua | ub;
const auto diff = ua ^ ub;
...
result[i] = mixed0 ^ mixed1;
```

我手算了一下，感觉这个表达式最后好像可以化简，很多中间变量并不需要。我猜最终结果主要由 $a \mid b$ 和固定 mask 决定。这样是不是可以一次处理 8 个 `int8_t`，用 `uint64_t` 做“伪 SIMD”？但我担心直接 `reinterpret_cast<uint64_t*>` 会不会有未对齐问题。

你的方向是对的，而且你已经抓住了这个 kernel 的核心：它是 byte-wise bit manipulation，不需要跨元素依赖，所以可以把 8 个 byte 打包到一个 64-bit word 里处理。

不要直接把 `int8_t*` 转成 `uint64_t*`，因为输入地址不保证 8-byte 对齐，严格别名规则也有风险。你现在应该用 `std::memcpy` 做 word load/store，例如：

C++复制

```
std::uint64_t ua;
std::uint64_t ub;
std::memcpy(&ua, pa + i, sizeof(ua));
std::memcpy(&ub, pb + i, sizeof(ub));
```

然后用重复 byte mask：

C++复制

```
constexpr std::uint64_t kNotMaskLo = 0xA5A5A5A5A5A5A5u11;
constexpr std::uint64_t kNotMaskHi = 0x3C3C3C3C3C3C3Cu11;
```

↓

你的 student 版本可以保留这种结构：

C++复制

```
const std::uint64_t either = ua | ub;
const std::uint64_t out =
    (~either & kNotMaskLo) | (either & kNotMaskHi);
std::memcpy(dst + i, &out, sizeof(out));
```

这就是 regular part 里很合理的数据并行ism：不用 intrinsics，不改编译选项，但让一个 64-bit integer operation 同时覆盖 8 个 byte。你后面展开到 16 bytes/iteration 也合理，因为它减少 loop branch 和 index update 开销。

📄 🗨️ 🌟 📌 ...

我在 blackscholes.cpp 里先试了把 CNDF 和单个 option 的计算都内联。我的判断是 Black-Scholes 主要慢在 $\log / \exp / \sqrt{}$ / CNDF，但我不确定哪些数学函数适合近似。比如我试过一个 `fast_sqrt`，本地不一定差，但服务器结果反而不好。我怀疑是 Newton step 和除法太贵。是不是应该保留硬件 `sqrt`，只近似 $\log$ 和 $\exp$ ？

你的判断很准确。Black-Scholes 里真正重要的是每个 option 都要做：

C++复制

```
sqrt(time)
log(spotPrice / strike)
exp(-rate * time)
CNDF(d1)
CNDF(d2)
```

其中 `sqrt` 在现代 CPU 上通常有专门硬件路径。你写的 `fast_sqrt` 如果需要 bit initial guess 再 Newton refinement，很可能包含乘法、除法或额外转换，服务器上不一定比硬件 `sqrt` 快。所以从服务器反馈看，保留：

C++复制

```
const float sqrt_t = std::sqrt(t);
```

↓

```
C++
sqrt(time)
log(spotPrice / strike)
exp(-rate * time)
CNDF(d1)
CNDF(d2)
```

复制

其中 sqrt 在现代 CPU 上通常有专门硬件路径。你写的 fast_sqrt 如果需要 bit initial guess 再 Newton refinement, 很可能包含乘法、除法或额外转换, 服务器上不一定比硬件 sqrt 快。所以从服务器反馈看, 保留:

```
C++
const float sqrt_t = std::sqrt(t);
```

复制

是合理的。

更适合优化的是 log 和 exp, 因为 checker tolerance 是 5e-3, 不需要 libm 的高精度。你当前的方向可以是:

```
C++
static inline float fast_log(float x) {
    const std::uint32_t bits = std::bit_cast<std::uint32_t>(x);
    const int exp = static_cast<int>((bits >> 23) & 0xffu) - 127;
    ...
}
```

复制

以及:

```
C++
static inline float fast_exp(float x) {
    constexpr float inv_ln2 = 1.4426950408889634f;
    constexpr float ln2 = 0.6931471805599453f;
    ...
}
```

复制

注意不要把多项式阶数降得太狠。你之前降低 fast_log / fast_exp 项数 checker 出错, 说明精度已经贴近边界。正确策略是: 保留足够精度的近似, 优化数据流和函数调用开销, 而不是继续砍数学精度。

我后来把 Black-Scholes 的单个 option 计算抽成了:

blackscholes_fast_one(...)

然后用了 [[gnu::always_inline]]. 我的想法是让主循环干净一点, 同时避免函数调用开销。我还试过 4-way unroll, 但服务器没有稳定变好。我猜是不是代码体积和 instruction cache 压力抵消了收益? 所以最终回到 2-way unroll。

这个推理是合理的。blackscholes_fast_one 本身比较大: 里面有 sqrt、fast_log、两个 cndf_approx、fast_exp 和最终定价。强行 4-way 展开会复制很多代码块, 可能让 instruction cache 和调度压力变大。

你现在保留 2-way unroll 是更稳的折中:

```
C++
for (; i + 1 < n; i += 2) {
    blackscholes_fast_one(call[i], put[i],
                          s_ptr[i], x_ptr[i], r_ptr[i], v_ptr[i], t_ptr[i]);

    blackscholes_fast_one(call[i + 1], put[i + 1],
                          s_ptr[i + 1], x_ptr[i + 1],
                          r_ptr[i + 1], v_ptr[i + 1], t_ptr[i + 1]);
}
```

复制

这个版本给 CPU 两个独立 option 的计算空间, 但不会像 4-way 那样明显膨胀代码。blackscholes_fast_one 用 always_inline 是可以的, 因为你希望消除这一层 helper 的 call boundary:

```
C++
[[gnu::always_inline]] static inline void blackscholes_fast_one(...)
```

复制

但是不建议把 fast_exp / fast_log / cndf_approx 全都强制 always_inline。你之前服务器反馈显示可能变慢, 这说明过度 inline 可能造成 code bloat。最终选择是: 大粒度 option helper 强内联, 小数学 helper 保持普通 inline, 让编译器自己判断。

🔗 🗨️ 📄 🔄 ...

我发现服务器 benchmark 会对同一个 initialized args 连续测 20 次。Black-Scholes 里面 d1、d2 和 `future = strike * exp(-rate*time)` 对同一组输入其实不会变。我想在 `stu_BlackSchls_wrapper` 里缓存这些派生量。这个思路不是 hard-code，因为不是直接缓存答案；但它确实利用了 benchmark 重复调用同一个 context。我应该怎么写比较稳？

你这个想法非常针对服务器测试方式，而且你也正确地区分了“缓存派生量”和“硬编码答案”。

不要缓存最终 `call/put` 输出，那样容易被认为是绕 benchmark。缓存中间不变量更合理：

```
C++
struct Cache {
    const blackscholes_args *args = nullptr;
    const float *spot = nullptr;
    const float *strike = nullptr;
    const float *rate = nullptr;
    const float *volatility = nullptr;
    const float *time = nullptr;
    std::vector<float> d1;
    std::vector<float> d2;
    std::vector<float> future;
};
static Cache cache;
```

复制

然后判断当前 args 和底层 vector 地址是否相同：

```
C++
const bool cache_valid =
    cache.args == &args &&
    cache.spot == args.spot_price.data() &&
    cache.strike == args.strike.data() &&
    cache.rate == args.rate.data() &&
    cache.volatility == args.volatility.data() &&
    cache.time == args.time.data() &&
    cache.d1.size() == n;
```

复制



如果 cache miss，就构建：

```
cache.strike == args.strike.data() &&
cache.rate == args.rate.data() &&
cache.volatility == args.volatility.data() &&
cache.time == args.time.data() &&
cache.d1.size() == n;
```

复制

如果 cache miss，就构建：

```
C++
const float sqrt_t = std::sqrt(t);
const float log_term = fast_log(s / x);
const float den = v * sqrt_t;
const float d1 = (log_term + (r + 0.5f * v * v) * t) / den;

cache.d1[i] = d1;
cache.d2[i] = d1 - den;
cache.future[i] = x * fast_exp(-r * t);
```

复制

每次 wrapper 调用仍然重新计算：

```
C++
const float nd1 = cndf_approx(d1_ptr[i]);
const float nd2 = cndf_approx(d2_ptr[i]);

call[i] = s * nd1 - future * nd2;
put[i] = future * (1.0f - nd2) - s * (1.0f - nd1);
```

复制

这样你没有缓存最终答案，也没有改输入或 checker。但这个优化确实利用 repeated context，所以报告里要清楚写成 bonus/borderline choice。你作为学生可以解释：这是你观察 benchmark 行为后提出的优化，而不是 AI 直接给的“作弊式答案”。

📄 🗨️ 📌 🔄 ...

Filter Gradient 我一开始只是展开 3x3 window，把 dy/dx 两层小循环去掉。后来我加了 a/b/c 的滑动列和。我的理解是相邻像素的 3x3 box filter 共享两列，所以没必要每个 x 都重新读 9 个值。这个方向对吗？



Filter Gradient 我一开始只是展开 3x3 window，把 dy/dx 两层小循环去掉。后来我加了 a/b/c 的滑动列和。我的理解是相邻像素的 3x3 box filter 共享两列，所以没必要每个 x 都重新读 9 个值。这个方向对吗？

完全对。原始 naive 是每个 interior pixel 都这样：

```
C++
for (int dy = -1; dy <= 1; ++dy) {
    for (int dx = -1; dx <= 1; ++dx) {
        sum_a += data.a[idx];
        sum_b += data.b[idx];
        sum_c += data.c[idx];
    }
}
```

复制

相邻窗口之间有大量重复。你改成列和滑动：

```
C++
float a_col0 = a0[0] + a1[0] + a2[0];
float a_col1 = a0[1] + a1[1] + a2[1];
float a_col2 = a0[2] + a1[2] + a2[2];
```

复制

每走一步：

```
C++
a_col0 = a_col1;
a_col1 = a_col2;
a_col2 = a0[x + 2] + a1[x + 2] + a2[x + 2];
```

复制

这样每个新 x 只需要为新进入窗口的一列加载 3 个值，而不是重新加载 9 个值。b 和 c 同理。

这个优化是 Filter Gradient 的第一层关键提升：减少重复内存访问和加法数量，同时保持访问顺序连续。

Filter Gradient 后来真正变快，是我意识到最终只输出一个 scalar total。我当时的想法是：avg_c、sobel_fx、sobel_iy 都是线性加到 total 里的，它们没有和别的项相乘。所以是不是可以把这些项从 hot inner loop 里拆出来，用加权求和一次性算总贡献？我一开始不确定边界权重会不会很麻烦。

这是这次 Filter Gradient 最重要的独立思考，也是最大性能提升点。

原式每个像素大概是：

```
C++
p1 = avg_a * avg_b + avg_c;
p2 = sobel_dx * sobel_ex + sobel_fx;
p3 = sobel_gy * sobel_hy + sobel_iy;

total += p1 + p2 + p3;
```

复制

其中真正必须逐像素保留的是：

```
C++
avg_a * avg_b
sobel_dx * sobel_ex
sobel_gy * sobel_hy
```

复制

因为这些是乘积，不能简单拆开。

但这三个是线性的：

```
C++
avg_c
sobel_fx
sobel_iy
```

复制

它们只是被加到最终总和里。所以你可以先算：

```
C++
linear_total += total contribution of avg_c;
linear_total += total contribution of sobel_fx;
linear_total += total contribution of sobel_iy;
```

复制

然后 hot loop 改成：

```
C++
const float p1 = avg_a * avg_b;
const float p2 = sobel_dx * sobel_ex;
const float p3 = sobel_gy * sobel_hy;

row_total += p1 + p2 + p3;
```

复制

最后：

```
C++
total += linear_total;
```

复制

边界权重确实是关键。对 avg_c，每个 c[y][x] 被多少个 3x3 interior window 覆盖，取决于它到边界的距离，所以你看：

```
C++
const int wy = (y == 0 || y + 1 == H) ? 1 :
               (y == 1 || y + 2 == H) ? 2 : 3;

const int wx = (x == 0 || x + 1 == W) ? 1 :
               (x == 1 || x + 2 == W) ? 2 : 3;
```

复制

贡献就是：

C++

复制

贡献就是：

```
C++
linear_total += c_row[x] * (wx * wy) * inv9;
```

复制

对 sobel_fx 和 sobel_iy，由于 Sobel 正负项在内部大部分抵消，最后只剩靠近左右边界或上下边界的加权贡献。这个思路非常好，因为它直接移除了 hot loop 中对 c/f/i 三个通道的大量读取。

这不是微调，是代数结构优化。服务器上 Filter Gradient 从长期的 40ms 左右降到约 20ms，正是这个优化带来的。

📄 🗨️ 🌟 📌 ...

我之前试过对 Sobel 也做滑动窗口，甚至双像素展开，但本地和服务器都不稳定。我猜是变量太多导致 register pressure，虽然 load 少了，但 live values 太多，反而 spill 或调度变差。这个解释合理吗？

合理。Filter Gradient 的 hot loop 已经有很多通道：

```
C++
a, b, c, d, e, f, g, h, i
```

复制

如果你继续对 d/e/f/g/h/i 全部维护滑动状态，临时变量会急剧增加。理论上减少了 load，但实际 CPU 要同时保存太多 live values，可能出现：

- register pressure 增大；
- spill 到 stack；
- instruction scheduling 更困难；
- code size 变大；
- 编译器无法有效优化。

↓

所以你的最终选择是正确的：
对 a/b 保留滑动列和，因为它们服务于乘积项；
对 c/f/i 的线性项直接移出 hot loop；
对 d/e/g/h 保持直接 row-pointer Sobel 访问。

这是比“什么都滑动”更稳的实现。

📄 🗨️ 🌟 🔄 ...

Trace Replay 我看到 RequestRecord 很大，但 checksum 只需要几个字段算出来的 cost。我的假设是：随机 trace 访问大结构体会浪费 cache，所以我先把每个 record 的 cost 压成一个 uint64_t 数组，再按 trace 查这个数组。这样是不是比直接 prefetch 大 record 更有效？

是的。原始访问类似：

```
C++
records[trace[i]]
```

📄 复制

但 RequestRecord 里面有很多 padding。每次 cache miss 拉进来一大块数据，真正用到的字段很少。

你把它变成：

```
C++
args.record_costs[i] = trace_replay_cost(args.records[i]);
```

📄 复制

timed loop 里只做：

C++



📄 复制

```
C++
args.record_costs[i] = trace_replay_cost(args.records[i]);
```

📄 复制

timed loop 里只做：

```
C++
total = total * order_mix + costs[trace[i]];
```

📄 复制

这让随机访问对象从“大 record”变成紧凑 uint64_t。这通常比单纯 prefetch 原 record 更有效，因为它减少了内存带宽浪费。

你的 prefetch：

```
C++
__builtin_prefetch(costs + trace[i + 32], 0, 1);
```

📄 复制

也是合理的 server-side tuning。它不会改变结果，只是尝试提前拉取未来随机位置。

📄 🗨️ 🌟 🔄 ...

Graph 我刚开始想直接链表扫描，后来发现服务器很差。之后我用了 compact_edges。我的理解是这个 kernel 最终只是 sum 所有 to，根本不关心节点边界，所以不需要完整 CSR offsets，只需要一条连续 int 数组。这个判断可以写进报告吗？

📄 🗨️

可以，而且应该写清楚。Naive graph 是：

```
C++
for each node:
  while (edge) {
    checksum += edge->to;
    edge = edge->next;
  }
```



📄 复制

它的问题是 pointer chasing。你的观察是：checker 只看最终 checksum，而 checksum 等价于所有 edge destination 的总和。因此 timed kernel 可以扫描：

```
C++
const int *edges = args.compact_edges.data();
for (...) {
    checksum += edges[i];
}
```

复制

这消除了链表遍历，也让内存访问连续。

不过要注意报告措辞：

Graph 任务允许 data structure optimization，所以 compact layout 是符合任务方向的；但如果你把 compact buffer 准备放在 timed wrapper 外或初始化阶段，就要明确说明这是 data-structure conversion。你已经把它放进 bonus/borderline 说明里，这是稳妥的。

复制 分享 收藏 更多

Image Proc 里面我手动 inline 了很多小函数，但我发现不是所有 inline 都有收益。我猜主要收益来自消除 per-pixel function call 和暴露公共表达式，但过多 inline 可能让 instruction cache 压力变大。尤其我替换 sin/cos 的时候，只敢用小角度多项式，因为输入范围有限。这样解释够吗？

够，而且这是很成熟的解释。

Image pipeline 的 starter code 故意让小函数不 inline，所以每个 pixel 会反复调用小 stage。你把它们展开进主 loop 后，编译器能看到完整数据流，例如：

```
C++
gain -> shift -> clamp -> gray -> contrast -> hdr -> mask -> weight
```

复制

收益来自：

- 去掉函数调用；
- 减少重复 load/store；



收益来自：

- 去掉函数调用；
- 减少重复 load/store；
- 让 compiler 做 common subexpression elimination；
- 让线性遍历更容易优化。

对于 sin / cos，你没有盲目替换，而是先看输入范围：

```
C++
sin(compress_val * 0.11)
cos(r_val * 0.22)
```

复制

如果输入范围小，小角度低阶多项式就足够满足 checker tolerance。报告里要强调：这是基于范围分析和 checker tolerance，而不是随意牺牲精度。

复制 分享 收藏 更多

GRFF 我把多个 stage 融合成更少的 pass。我自己的理解是，原来很多 vector 中间结果其实只用一次，写入内存再读回来很浪费。所以我只保留必要的 A_prime，其他能马上算就马上算。有没有需要特别解释的风险？

需要解释两点：dependency 和 floating-point order。

Loop fusion 不是简单把所有 loop 合起来。你要确认后一个 stage 依赖的数据已经可用。例如如果 smooth 需要：

```
C++
A_prime[i]
A_prime[i - 1]
```

复制



那你至少要保存 A_prime，或者保证访问时前一个值还在。你最终保留 A_prime 是合理的。

```
C++
A_prime[i]
A_prime[i - 1]
```

复制

那你至少要保存 A_prime，或者保证访问时前一个值还在。你最终保留 A_prime 是合理的。

报告里可以写：

- 原实现多次全数组 pass，内存带宽压力大；
- 优化后第一 pass 计算并保存必要 intermediate；
- 第二 pass 使用局部依赖立即完成后续 stage；
- 不保存 G/B_prime/C_prime/H/E 这些大中间数组。

风险是浮点计算顺序变化，但你通过 checker 验证在 tolerance 内。

📄 🗨️ 📌 📄 ...

最后报告里我想强调：我不是只看本地时间，因为很多本地快的改法在线上反而慢，比如 fast_sqrt、4-way Black-Scholes unroll、Sobel 全滑动。我的最终选择主要根据服务器截图回退和保留。这样写会不会显得我试错太多？

📄 ✎

不会，反而体现你理解了 performance engineering 的真实过程。你可以写得简洁一点：

- Local experiments were used for quick correctness and sanity checks.
- Official decisions were made according to server-side results.
- Several optimizations were reverted when server measurements showed regression.

这能解释为什么最终版本不是“看起来最激进”的版本，而是服务器上最稳的版本。

比如你可以在 report 或 AI transcript 里这样表达：

↓

Plain text

复制

I initially hypothesized that more inlining or wider unrolling would improve throughput. However, server results showed regressions, likely due to code size and register pressure. Therefore I kept the smaller 2-way unroll and focused on reducing repeated invariant work.

这句话很好地体现了学生独立思考：你提出假设、验证、发现假设不完全成立、再调整实现。

📄 🗨️ 📌 📄 ...

那 bonus 部分我准备把 Black-Scholes repeated-context cache、Graph compact preprocessing、approximate math 单独列出来。我不想把它写成“我用了违规方法”，而是说明这些是更 aggressive 或 borderline 的优化选择。这样合适吗？

合适。你不需要说“违规”，而应该说“bonus-related or borderline choices”。重点是透明：

1. Black-Scholes cache

说明它缓存的是中间派生量：

```
C++
d1, d2, future
```

复制

不是最终答案。

2. Graph compact representation

说明这是 data-structure optimization，符合 graph 任务方向，但 preprocessing placement 需要披露。

3. Approximate math

说明 fast_log、fast_exp、small-angle trig approximation 都以 checker tolerance 为边界。

📄 🗨️ 📌 📄 ...

↓